

There are many others, and at https://wiki.postgresql.org/wiki/Binary_Replication_Tools, you can find a good comparison of them all.

In this section, we will give a nod to pgbackrest; it is one of the most used tools for disaster recovery because it allows, in a very simple way, scalability on cores – and it allows the possibility of saving data on buckets in a compressed and encrypted way. The pgbackrest tool is a tool for PostgreSQL disaster recovery and PITR, and it has been designed for heavy load servers. Its official URL is <https://pgbackrest.org/>.

These are some of the features of the tool:

- It supports parallel backup and parallel restore.
- It can make full base backups, incremental backups, or differential backups.
- We can choose to do local operations or remote operations.
- We can choose our policy retention for backups and archive expiration.
- It supports backup resume.
- It supports streaming compression and checksums.
- For a restore procedure, we can use the delta restore feature.
- It is possible to use parallel WAL archiving.
- It supports tablespaces and links.
- It supports data encryption.
- It supports SFTP support for repository storage.
- It supports object store storage for S3, GCP, and Azure.

Basic concepts

The pgbackrest tool uses the concept of stanzas, and it can use a local repository or an external repository:

- A stanza is a configuration of a remote server for backup. It is a set of targets to be backed up. A stanza configuration can contain multiple servers, in which case the first (pg1) is the master, and the others are considered standby servers.
- A repository is local or remote storage (SSH) to which backups are saved; it can be encrypted. A repository can contain multiple definitions, but only the first one (repo1) is currently supported.

- It is important to have a public key exchange between users who use pgbackrest. The simplest thing to do is to have public keys exchanged between the Postgres user of the PostgreSQL server and the Postgres user of the server where the pgbackrest repository is present.

Environment set up

For this section, there is no Docker container, and so before starting and testing our pgbackrest tool, let's see what we need to start working. We will need the following things:

- A running PostgreSQL server
- A server where we will install and configure the pgbackrest tool with a postgres user

In this scenario, we will continue to use our pg1 PostgreSQL server with ip= 192.168.122.170. We also need to add another server called pgbackrest with an IP address of 192.168.122.120.

The exchange of public keys

We will now see how to exchange public keys before we install pgbackrest:

1. First of all, let's create an ssh key for the Postgres user on both servers. As a PostgreSQL user, let's execute the following:

```
postgres@pgbackrest:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/postgres/.ssh/id_rsa):
Created directory '/home/postgres/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/postgres/.ssh/id_rsa
Your public key has been saved in /home/postgres/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:5BPkarhop6Z82WeWtYM1i5gHseFHAVJEoKy8GF5HjQ postgres@
pgbackrest
The key's randomart image is:
+---[RSA 4096]---+
| oE. oo+=.      |
|+ooo. o+o      |
|o=. .   o+.     |
|. .   ..+o.     |
```

```

|      .+o=S.      |
|      .oo= =.      |
|      o =. +.+      |
|      ...= .o=.      |
|      .+o  . =      |
+-----[SHA256]-----+

postgres@pg1:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/postgres/.ssh/id_rsa):
Created directory '/home/postgres/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/postgres/.ssh/id_rsa
Your public key has been saved in /home/postgres/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:g/amWaxcTGsmx2WQ91U/23UcBXmDtSsfRYhqE7dMWko postgres@pg1
The key's randomart image is:
+---[RSA 4096]-----+
|          .+*=|
|          .E =.o==|
|          o..X ..+B|
|          . o*.o.  O|
|          o S.o... +.|
|          . * =   o .|
|          . &      . |
|          . %      |
|          =        |
+-----[SHA256]-----+

```

When we execute the `ssh-keygen` command, we have to make sure to only press the *Enter* key at the request of the passphrase.

2. Now, on both servers, we will have two files in the `~/ .ssh` directory of the `postgres` user:

```

postgres@pgbackrest:~/ .ssh$ ls -l
total 8
-rw----- 1 postgres postgres 3389 Jul 10 08:34 id_rsa
-rw-r--r-- 1 postgres postgres  745 Jul 10 08:34 id_rsa.pub

```

```
postgres@pg1:~/.ssh$ ls -l
total 8
-rw----- 1 postgres postgres 3381 Jul 10 08:34 id_rsa
-rw-r--r-- 1 postgres postgres  738 Jul 10 08:34 id_rsa.pub
```

3. The fastest way to exchange public keys between the two servers is using the `ssh-copy-id` command:

```
postgres@pg1:~/.ssh$ ssh-copy-id 192.168.122.120
[.. cutted..]
Number of key(s) added: 1

Now try logging into the machine, with:  "ssh '192.168.122.120'"
and check to make sure that only the key(s) you wanted were added.

postgres@pgbackrest:~/.ssh$ ssh-copy-id 192.168.122.170
[.. cutted..]

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh '192.168.122.170'"
and check to make sure that only the key(s) you wanted were added
```

Now, using the `postgres` user, it is possible to connect the two servers together without providing a password.

Installing pgbackrest

Before installing, we have to check if the same version of `pgbackrest` is present in the repository of each host. After checking this, let's install it on the `pgbackrest` server and on the `pg1` server. On a Debian-like server, as the `root` user, let's execute these commands on both servers:

```
root@pg1:~# apt-get update
root@pgbackrest:~# apt-get update

root@pg1:~# apt-get install pgbackrest
root@pgbackrest:~# apt-get install pgbackrest
```

If we use a RHEL server, we have to use the `yum` command instead of the `apt-get` command.

Configuring pgbackrest

Now let's look at how to configure the pgbackrest tool. It needs the configuration on both servers; it needs the configuration of the repository server, which is where the data will be stored, and it needs the configuration of the PostgreSQL server so that it is able to send all the data to the repository server. So, we will address both of these configurations in turn:

- The repository configuration of the pgbackrest server
- The PostgreSQL configuration of the pg1 server

The repository configuration

The configuration file for the pgbackrest server can be found here:

```
/etc/pgbackrest.conf
```

Using a different configuration file is possible but this must be specified consistently in each use of the program, so it's better to leave the default one. Each parameter specified in the configuration file can be overwritten by the relative parameter provided on the command line. Each parameter contained in a section is specified with a key-value pair. In the stanza configuration, the parameters of a cluster always start with pgN-, with N being a progressive number. The main (primary) cluster is always number 1. The standby clusters are therefore numbered in sequence, starting from number 2. Similarly, in global parameters, repositories are numbered starting from 1 (repo1), but currently, multiple repositories are not supported. pgbackrest is symmetric; that is, every command can be executed on the backup machine or on the target machine. We will have a configuration file for the repository server and a configuration file for the PostgreSQL server, and the two configuration files are different. pgbackrest, by default, has enabled the compression of WAL segments and base backups with a compression factor of 6. We can force the compression to a different level using the compress-level directive; for example, we can set the compression level to 9 to have the maximum compression.

It is also possible to encrypt the repository managed by pgbackrest; this feature is useful for storing our backups on a low-cost cloud, for example.

Let's start now with a simple configuration; let's start with the global configuration section on the pgbackrest server:

```
[global]
start-fast=y
archive-async=y
```

```
process-max=2
repo-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-archive=5
repo1-retention-diff=3
log-level-console=info
log-level-file=info
```

We see the following options here:

- `start-fast=y`: Forces a checkpoint on the remote server, so that `pg_start_backup ()` starts as soon as possible.
- `archive-async=y`: Enables the asynchronous transfer of WAL for push/pull operations.
- `process-max=2`: Sets the maximum number of processes that the system can use for transfer/compression operations.
- `repo-path=/var/lib/pgbackrest`: Sets the path where the repository will be stored; the user running the `pgbackrest` command must have read/write permissions for this directory.
- `repo1-retention-full=2`: The number of full backups to keep. When a full backup expires, all differential and/or incremental backups associated with the full backup will also expire. When the option is not defined, the system issues a warning. If indefinite retention is desired, set the option to the maximum value (9,999,999).
- `repo1-retention-archive=5`: Represents the backup number of the WAL files to keep. The WAL segments required to make a backup consistent are always maintained until the backup expires, regardless of the configuration of this option. If this value is not set, the expiring archive will automatically expire at the `repo-retention-full` (or `repo-retention-diff`) value corresponding to the type of `repo-retention` archive if set to `full` (or `diff`). This will ensure that the WAL files are considered expired only for backups that have already expired.
- `repo1-retention-diff = 3`: The number of differential backups to keep. When a differential backup expires, all incremental backups associated with the differential backup will also expire. If not defined, all differential backups will be kept until the full backups on which they depend expire.
- `log-level-console=info/log-level-file=info` `log`: Settings for log management; set the terminal log level (`log-level-console`) and the logging level on the log file (`log-level-file`).

The configuration file shown here is just a simple example; if we want to add some more features, we just need to add them to the configuration file.

For example, if we want to modify the compression level and increase it to level 9, we can add these lines:

```
compress = y
compress-level = 9
compress-level-network = 9
```

In the same way, if we want to add the cipher feature, we can add these lines:

```
repo1-cipher-type = aes-256-cbc
repo1-cipher-pass = LearnPostgreSQL
```

After configuring the global section, we are ready to look at how to configure the stanza. `pgbackrest` introduces the idea of stanzas; in practice, we can associate each stanza with a cluster database. The following is an example of a room; it is only a coincidence that the name of the stanza, `[pg1]`, has the same name as the cluster. It is necessary to create a stanza for each remote PostgreSQL server on which we want to manage backups using `pgbackrest`. Each stanza must have a different name:

```
[pg1]
pg1-host = 192.168.122.170
pg1-host-user = postgres
pg1-path = /var/lib/postgresql/16/main
pg1-port = 5432
```

We see the following options here:

- `pg1-host`: This is the remote host of the PostgreSQL master server.
- `pg1-host-user = postgres`: When the `pg-host` parameter is set, this is the user that we want to use to access the remote PostgreSQL server. This user will also be the owner of the remote `pgbackrest` process and it starts the connection to the PostgreSQL server. This user should be the owner of the PostgreSQL database cluster. Usually, we can leave the default user, `postgres`, which is why it is usually the same user for whom we made the exchange of public keys.
- `pg1-path = /var/lib/postgresql/16/main`: The path on the PostgreSQL cluster where the data is stored. We can find it in the `data_directory` parameter inside the `postgresql.conf` file.

- `pg1-port = 5432`: The listen port of the remote PostgreSQL server.

Using pgbackrest with object store support

pgbackrest supports object stores for Azure, GCP, and Amazon S3; that means that pgbackrest is available to store all the data directly on a low-cost bucket using the data encryption we've seen before.

The configuration is pretty simple; for example, for a GCP bucket, we have to specify the parameters below:

```
repo1-type=gcs
repo1-path=/path_on the bucket
repo1-gcs-bucket=bucket_name
repo1-gcs-key=/etc/pgbackrest-key.json
```

Adding those parameters, pgbackrest will be able to store all data in a Google bucket; this feature is very useful for the DBA because it allows us to store and encrypt our data on the cloud at a small price.

In this way, we don't have to worry about the size of the disk that contains the repository used by pgbackrest and, at the same time, since the cost per GB per bucket is very low, we can greatly increase our retention. For further information about the pgbackrest configuration with S3, Azure, and GCP buckets, you can visit the links below:

- <https://pgbackrest.org/user-guide.html#azure-support>
- <https://pgbackrest.org/user-guide.html#s3-support>
- <https://pgbackrest.org/user-guide.html#gcs-support>

The PostgreSQL server configuration

Now let's move on to the PostgreSQL server configuration. On the pg1 server, we need to modify the `postgresql.conf` file and we need to set the `pgbackrest.conf` file as well.

The postgresql.conf file

For the `postgresql.conf` file, we have to set `wal_level` to `replica` or `logical`. It is important that the WAL level is not set to `minimal`, because PITR is not possible if `wal_level=minimal`. We also need to tell PostgreSQL the command that will send the WAL segment to the pgbackrest repository server.

Let's add these lines at the end of the `postgresql.conf` file:

```
#PGBACKREST
archive_mode = on
wal_level = replica #logical if we have some logical replications
archive_command = 'pgbackrest --stanza=pg1 archive-push %p'
```

With the second line, we say to PostgreSQL that the WAL segments will be archived on the `pg1` stanza of the repository server using the `pgbackrest` command. After restarting PostgreSQL, these new lines will be available. As the root user, let's perform a restart of the PostgreSQL service:

```
# systemctl restart postgresql
```

The `pgbackrest.conf` file

Now, after modifying `postgresql.conf`, let's go to modify the `pgbackrest.conf` file of the PostgreSQL server. Let's remember that the PostgreSQL server has `ip= 192.168.122.170`, and that the IP of the disaster recovery server is `192.168.122.170`. Let's now edit the `/etc/pgbackrest.conf` file; delete what is present and add these lines:

```
[global]
backup-host=192.168.122.120
backup-user=postgres
backup-ssh-port=22
log-level-console=info
log-level-file=info

[pg1]
pg1-path = /var/lib/postgresql/16/main
pg1-port = 5432
```

As for the repository configuration, the file is composed of sections: a `global` section and a section for each stanza.

For the `global` section, we have the following options:

- `backup-host`: The repository host
- `backup-user`: The user used for the backup
- `backup-ssh-port`: The ssh port
- `log-level-console=info` and `log-level-file=info`: As we've seen in the previous section

For the stanza section, we have the following options:

- `pg1-path = /var/lib/postgresql/16/main`: The path on the PostgreSQL cluster where the data is stored. We can find it in the `data_directory` parameter inside the `postgresql.conf` file.
- `pg1-port = 5432`: The listen port of the remote PostgreSQL server.

Creating and managing continuous backups

Now that we have our system well configured, let's start to manage our backups.

Creating the stanza

The first thing we have to do is create the stanza on the repository server. To do this, as a postgres user, let's perform this command:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 stanza-create
2023-07-11 08:34:48.439 P00    INFO: stanza-create command begin 2.46:
--compress-level-network=9 --exec-id=2602-fbad976e --log-level-
console=info --log-level-file=info --pg1-host=192.168.122.170 --pg1-host-
user=postgres --pg1-path=/var/lib/postgresql/16/main --pg1-port=5432
--repo1-cipher-pass=<redacted> --repo1-cipher-type=aes-256-cbc --repo1-
path=/var/lib/pgbackrest --stanza=pg1
2023-07-11 08:34:49.558 P00    INFO: stanza-create for stanza 'pg1' on
repo1
2023-07-11 08:34:49.741 P00    INFO: stanza-create command end: completed
successfully (1305ms)
```

Now our stanza is created. If we go to `/var/lib/pgbackrest`, we can find the directory structure that will be used by the continuous backup system:

```
postgres@pgbackrest:/var/lib/pgbackrest$ ls -l
total 8
drwxr-x--- 3 postgres postgres 4096 Jul 11 08:34 archive
drwxr-x--- 3 postgres postgres 4096 Jul 11 08:34 backup
```

Checking the stanza

After creating our stanza, let's check whether the system is ready to accept the continuous backup by performing this:

```
postgres@pgbackrest:~$ pgbackrest --stanza=pg1 check
```